SQL & Python Interview Questions — Architect-Level Answers

Each answer covers the working solution *and* the design trade-offs a data/software architect would flag: performance at scale, edge cases, and when to reach for a different tool entirely.

PART 1 — SQL Query-Based Questions

1. Find the 2nd, 3rd, and Nth highest salary

```sql
SELECT salary
FROM (
  SELECT salary,
         DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
) t
WHERE rnk = 2;    -- swap 2 -> 3 -> N as needed
```

Architect notes - Use `DENSE_RANK()`, not `RANK()`, if ties should share a position (e.g., two people tied for 2nd both count as 2nd, and the next distinct salary is 3rd). `RANK()` will skip numbers on ties; `ROW_NUMBER()` ignores ties entirely. - Avoid the classic `LIMIT 1 OFFSET N-1` or nested `MAX(salary WHERE salary < (SELECT MAX...))` pattern for "Nth" — it doesn't parameterize well and degrades badly past N=3-4 (O(N) subqueries). - For a fully parameterized "give me the Nth highest" as a stored procedure/function, pass N as a parameter into the `WHERE rnk = :N` form above — one query, one index scan on `salary`, regardless of N. - Index `salary` if this runs frequently on a large table; window functions still require a sort unless the index supports order.

---

2. Find duplicate records and delete only the duplicates (keep one copy)

```sql
-- Identify duplicates (based on business key columns, e.g., email)
SELECT email, COUNT(*)
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;

-- Delete all but one copy per duplicate group
WITH ranked AS (
  SELECT ctid,                                  -- or a surrogate PK
         ROW_NUMBER() OVER (
           PARTITION BY email
           ORDER BY created_at ASC              -- keep the oldest/earliest
         ) AS rn
  FROM customers
)
```

```sql
DELETE FROM customers
WHERE ctid IN (SELECT ctid FROM ranked WHERE rn > 1);
```

(ctid is Postgres-specific; use the primary key in MySQL/SQL Server, e.g., id IN (SELECT id FROM ranked WHERE rn > 1).)

Architect notes - Always define "duplicate" precisely — same primary key? Same natural/business key? Same across all columns? This changes the PARTITION BY clause entirely. - Decide the tie-breaker for which row survives (ORDER BY created_at, ORDER BY id) — never leave it to the DB's arbitrary row order. - On large tables, run this in **batches** (DELETE ... LIMIT n in a loop, or partitioned deletes) to avoid long lock waits and transaction log bloat. - In production, prefer a **soft-delete / archive-then-purge** pattern over a hard DELETE, so you can recover if the dedup logic was wrong. - Add a unique constraint after cleanup so duplicates can't recur — fixing data without fixing the root cause just means you're back here next month.

---

3. Top 3 highest-paid employees from each department

```sql
SELECT department, employee, salary
FROM (
  SELECT department, employee, salary,
         DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS
rnk
  FROM employees
) t
WHERE rnk <= 3;
```

Architect notes - DENSE_RANK vs ROW_NUMBER: if two people in a department are tied for 3rd, DENSE_RANK returns both (could give you 4 rows for "top 3"); ROW_NUMBER strictly caps at 3 rows by breaking ties arbitrarily (needs a deterministic ORDER BY tie-breaker, e.g., ORDER BY salary DESC, employee_id ASC). Pick based on business requirement, not habit. - This pattern (partition + rank + filter) is the general-purpose "top-N per group" idiom — memorize it once, apply everywhere (top products per category, latest N events per user, etc.). - For very large tables with many partitions, ensure an index on (department, salary) so the engine can do a partitioned sort efficiently rather than a full sort + spill to disk.

---

4. Calculate Month-over-Month (MoM) sales growth

```sql
WITH monthly AS (
  SELECT DATE_TRUNC('month', order_date) AS month,
         SUM(amount) AS total_sales
  FROM orders
  GROUP BY DATE_TRUNC('month', order_date)
)
SELECT month,
       total_sales,
```

```sql
       LAG(total_sales) OVER (ORDER BY month) AS prev_month_sales,
       ROUND(
         (total_sales - LAG(total_sales) OVER (ORDER BY month))
         / NULLIF(LAG(total_sales) OVER (ORDER BY month), 0) * 100, 2
       ) AS mom_growth_pct
FROM monthly
ORDER BY month;
```

Architect notes - `NULLIF(..., 0)` guards against divide-by-zero for a month with no prior sales. - Watch out for **missing months** (no orders at all) — `LAG` will silently skip a gap instead of treating it as zero. Generate a full calendar-month spine (see Q7) and `LEFT JOIN` sales onto it if gaps must show as 0% or NULL growth explicitly. - For dashboards refreshed frequently, materialize the monthly aggregate into a summary table updated incrementally, rather than re-aggregating the full `orders` table on every query.

---

5. Find customers who didn't place any orders in the last 90 days

```sql
SELECT c.customer_id, c.name
FROM customers c
WHERE NOT EXISTS (
  SELECT 1
  FROM orders o
  WHERE o.customer_id = c.customer_id
    AND o.order_date >= CURRENT_DATE - INTERVAL '90 days'
);
```

Architect notes - `NOT EXISTS` generally outperforms `NOT IN` (which has NULL-handling pitfalls — a single NULL in the subquery's result makes `NOT IN` return zero rows) and often outperforms a `LEFT JOIN ... WHERE o.id IS NULL` on large tables because the optimizer can short-circuit per row. - Confirm whether "no orders" means *never ordered* vs *ordered before but not recently* — the query above captures both; if you need to distinguish "new" customers from "churned" customers, add a check against `c.created_at`. - Index `orders(customer_id, order_date)` — this is the single most impactful index for this query pattern.

---

6. Find consecutive login days for each customer

```sql
WITH login_days AS (
  SELECT DISTINCT customer_id, login_date
  FROM logins
),
grouped AS (
  SELECT customer_id, login_date,
         login_date - (ROW_NUMBER() OVER (
             PARTITION BY customer_id ORDER BY login_date
         ))::int AS grp   -- "islands and gaps" trick
  FROM login_days
```

```
)
SELECT customer_id,
       MIN(login_date) AS streak_start,
       MAX(login_date) AS streak_end,
       COUNT(*) AS streak_length
FROM grouped
GROUP BY customer_id, grp
HAVING COUNT(*) > 1
ORDER BY customer_id, streak_start;
```

Architect notes - This is the classic **"gaps and islands"** problem. The trick: subtracting a sequential row number from a sequential date collapses consecutive dates onto the same constant (`grp`), letting `GROUP BY` isolate each streak. - Date arithmetic syntax differs by engine (`login_date - ROW_NUMBER...` works cleanly in Postgres with date subtraction; in MySQL/SQL Server, use `DATEADD`/`DATE_SUB` on the row number instead). - If "consecutive" needs to account for gaps of exactly one day allowed (e.g., weekends don't break a streak), the grouping key needs adjusting — clarify business rules before generalizing this pattern.

---

7. Identify missing dates in a transaction table

```
WITH calendar AS (
  SELECT generate_series(
    (SELECT MIN(txn_date) FROM transactions),
    (SELECT MAX(txn_date) FROM transactions),
    INTERVAL '1 day'
  )::date AS cal_date
)
SELECT cal_date
FROM calendar
LEFT JOIN transactions t ON t.txn_date = cal_date
WHERE t.txn_date IS NULL
ORDER BY cal_date;
```

Architect notes - The core pattern: **generate a complete date spine, then anti-join against real data.** This is the single most reusable trick for gap detection, reporting continuity, and SLA/uptime analysis. - `generate_series` is Postgres syntax. In SQL Server, use a recursive CTE or a numbers/tally table; in MySQL 8+, a recursive CTE with `DATE_ADD`. - For production reporting, maintain a permanent **calendar dimension table** (pre-built, joinable, supports fiscal periods/holidays) instead of generating date ranges ad hoc in every query.

---

8. Calculate a running total of sales by customer

```
SELECT customer_id, order_date, amount,
       SUM(amount) OVER (
         PARTITION BY customer_id
         ORDER BY order_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

```
       ) AS running_total
FROM orders
ORDER BY customer_id, order_date;
```

Architect notes - The explicit frame clause (`ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`) is technically the default for `ORDER BY` + `SUM() OVER`, but stating it explicitly avoids ambiguity across engines and documents intent for the next engineer. - Multiple orders on the **same date** with `ROWS` framing will still get distinct running totals per row (correct for a strict running total); if you instead want same-day orders to sum together, switch to `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. - Running totals over huge partitions can be expensive; if this feeds a dashboard, consider precomputing/caching per customer rather than recalculating on every read.

---

9. Pivot rows into columns (monthly sales report) and vice versa

Rows → Columns (Pivot):

```
SELECT
  product_id,
  SUM(CASE WHEN month = 'Jan' THEN sales ELSE 0 END) AS jan,
  SUM(CASE WHEN month = 'Feb' THEN sales ELSE 0 END) AS feb,
  SUM(CASE WHEN month = 'Mar' THEN sales ELSE 0 END) AS mar
FROM monthly_sales
GROUP BY product_id;
```

(Or `PIVOT` in SQL Server, `crosstab()` in Postgres with the `tablefunc` extension.)

Columns → Rows (Unpivot):

```
SELECT product_id, 'jan' AS month, jan AS sales FROM sales_wide
UNION ALL
SELECT product_id, 'feb', feb FROM sales_wide
UNION ALL
SELECT product_id, 'mar', mar FROM sales_wide;
```

(Or `UNPIVOT` in SQL Server; `LATERAL` / `unnest` in Postgres for a cleaner version.)

Architect notes - The `CASE WHEN` + `SUM` pivot is portable across every SQL engine — it's the safest default even though native `PIVOT`/`crosstab` syntax exists, because it avoids vendor lock-in and is easy to extend when a new column (month) appears. - Pivots with a **dynamic, unknown number of columns** (e.g., new product categories added monthly) can't be done in static SQL — that requires dynamic SQL (building the query string at runtime) or handling the reshape in the application/BI layer (Pandas `pivot_table`, Excel, Power BI) instead of the database. - Don't pivot inside the database if the result set is meant for further relational processing — pivoted/wide data is harder to filter, join, and index. Pivot at the last mile (reporting layer).

---

10. Compare two tables and find new, updated, and deleted records

```sql
-- Assuming key = id, comparing old_table vs new_table
-- NEW records: exist in new, not in old
SELECT n.*
FROM new_table n
LEFT JOIN old_table o ON n.id = o.id
WHERE o.id IS NULL;

-- DELETED records: exist in old, not in new
SELECT o.*
FROM old_table o
LEFT JOIN new_table n ON o.id = n.id
WHERE n.id IS NULL;

-- UPDATED records: exist in both, but values differ
SELECT n.*
FROM new_table n
JOIN old_table o ON n.id = o.id
WHERE n.updated_at <> o.updated_at   -- or compare a hash/checksum of all
columns
   OR n.status <> o.status
   OR n.amount <> o.amount;
```

Architect notes - For wide tables, comparing column-by-column in the `WHERE` clause is error-prone (easy to forget a column) and slow. Instead, compute a **row hash/checksum** (`MD5(CONCAT(col1, col2, ...))` or `HASHBYTES` in SQL Server) on both sides at write time and compare hashes — one condition instead of N. - This is exactly the pattern behind **CDC (Change Data Capture)** and **SCD Type 2** dimension loading in data warehousing. At scale, don't do this with ad hoc full-table joins on every run — use a proper CDC tool (Debezium, native DB replication logs, or a `last_modified` incremental watermark) so you're comparing deltas, not entire tables, every time. - `NULL`-safe comparisons matter: `n.status <> o.status` silently drops rows where either side is `NULL` (three-valued logic). Use `IS DISTINCT FROM` (Postgres) or `COALESCE`/`ISNULL` wrapping in engines that lack it.

PART 2 — Python Programming Questions

1. Common elements between two lists

```python
def common_elements(list1, list2):
    return list(set(list1) & set(list2))
```

Architect notes - `set` intersection is O(n + m) — vastly better than the naive nested-loop O(n·m). - This **loses ordering and duplicate counts**. If order or multiplicity matters (e.g., "common elements, preserving how many times each appears"), use `collections.Counter` intersection instead: `python  from collections import Counter  common = list((Counter(list1) & Counter(list2)).elements())` - If elements are unhashable

(e.g., dicts), sets won't work — fall back to a hashable key extraction or an explicit list comprehension with `in` (accepting the O(n·m) cost, or converting to tuples first).

---

2. Compare two CSV files and generate Insert, Update, Delete records

```
import pandas as pd

def diff_csv(old_path, new_path, key_col):
    old = pd.read_csv(old_path).set_index(key_col)
    new = pd.read_csv(new_path).set_index(key_col)

    inserts = new.loc[new.index.difference(old.index)]
    deletes = old.loc[old.index.difference(new.index)]

    common_ids = new.index.intersection(old.index)
    merged = old.loc[common_ids].compare(new.loc[common_ids])
    updates = new.loc[merged.index.get_level_values(0).unique()] if not
merged.empty else pd.DataFrame()

    return inserts, updates, deletes
```

Architect notes - `DataFrame.compare()` (pandas ≥1.1) cleanly surfaces only the changed cells — better than a manual column loop. - This whole approach assumes both files **fit in memory**. For files too large to load fully, stream both files sorted by key and do a merge-join in a single pass (like a SQL merge join) — O(n) memory instead of O(n) per file loaded whole. - Real-world CSV diffing needs to handle: dtype drift (a column read as `int` in one file, `float` in another due to a stray blank), whitespace/encoding differences, and floating-point equality (`0.1 + 0.2 != 0.3` style false "updates"). Normalize types and round floats before comparing.

---

3. Flatten a deeply nested JSON into a tabular format

```
def flatten_json(obj, parent_key='', sep='.'):
    items = {}
    if isinstance(obj, dict):
        for k, v in obj.items():
            new_key = f"{parent_key}{sep}{k}" if parent_key else k
            items.update(flatten_json(v, new_key, sep))
    elif isinstance(obj, list):
        for i, v in enumerate(obj):
            items.update(flatten_json(v, f"{parent_key}[{i}]", sep))
    else:
        items[parent_key] = obj
    return items
```

Usage: `pd.json_normalize(list_of_flattened_dicts)` to get a DataFrame, or use `pandas.json_normalize(data, sep='.')` directly for simpler nesting (it handles dict nesting natively but arrays need a `record_path`).

Architect notes - Recursive flattening is fine for moderate depth; for genuinely deep or cyclic-risk structures (e.g., untrusted external JSON), guard against **recursion depth limits** and consider an iterative stack-based flattener instead. - Arrays are the tricky part: flattening `list[i]` into columns works for fixed-length arrays but creates a **ragged schema** (different rows have different column counts) for variable-length arrays. Decide upfront: explode arrays into separate rows (normalized/long format — usually the right call for downstream SQL) vs. flatten into indexed columns (wide format — only works if array length is bounded and consistent). - For genuinely large/nested JSON at scale, this is a solved problem in tools like Spark (`explode` + schema inference) — don't hand-roll flattening for multi-GB JSON; use a distributed engine.

---

4. Remove duplicate records while keeping the latest

```
import pandas as pd

df = pd.read_csv("data.csv")
df_dedup = (
    df.sort_values("updated_at")
      .drop_duplicates(subset="record_id", keep="last")
)
```

Pure-Python version (streaming, no pandas):

```
latest = {}
for row in records:  # each row: dict with 'id' and 'updated_at'
    if row['id'] not in latest or row['updated_at'] >
latest[row['id']]['updated_at']:
        latest[row['id']] = row
result = list(latest.values())
```

Architect notes - The dict-keyed approach is O(n), single-pass, and works on a generator/stream — it doesn't require loading the whole dataset into a DataFrame, which matters once the file no longer fits comfortably in memory. - Sorting before `drop_duplicates` is the easy pandas idiom, but sorting is O(n log n) when the dict approach achieves the same result in O(n). Prefer the dict/streaming version for large files. - Watch for **tie-breaking on identical timestamps** — decide a deterministic secondary sort key (e.g., a monotonically increasing ingestion sequence number) so results are reproducible on reruns.

---

5. Count the frequency of every word in a large text file

```
from collections import Counter
```

```python
def word_frequency(filepath):
    counter = Counter()
    with open(filepath, 'r', encoding='utf-8') as f:
        for line in f:                          # stream line by line
            words = line.lower().split()
            counter.update(words)
    return counter
```

Architect notes - Reading **line by line** (not `f.read()` the whole file) keeps memory flat regardless of file size — this is the key "large file" adaptation. - `Counter.update()` is implemented in C and is materially faster than manually incrementing a dict in a Python loop. - For files too large for a single machine's memory/CPU budget in reasonable time, this is literally the canonical MapReduce example — split the file into chunks, count per chunk in parallel (multiprocessing or Spark), then merge `Counter` objects (`Counter` objects support + to merge). - Real-world word counting needs tokenization decisions: punctuation stripping, Unicode normalization, stopword removal — `line.split()` is a naive tokenizer; use `re.findall(r"\w+", line.lower())` for something more robust.

---

6. Merge multiple CSV files into a single output efficiently

```python
import pandas as pd
import glob

def merge_csvs(folder, output_path):
    files = glob.glob(f"{folder}/*.csv")
    with open(output_path, 'w') as out:
        for i, f in enumerate(files):
            df = pd.read_csv(f)
            df.to_csv(out, header=(i == 0), index=False, mode='a')
```

Architect notes - Writing with `mode='a'` and controlling the header only on the first file avoids ever holding all files in memory simultaneously (`pd.concat` of every file loaded at once does not scale). - Schema drift is the real-world killer here — if file 5 has a renamed or reordered column, this silently misaligns data. Validate column sets match before appending, or explicitly reindex each `df` to a canonical column order (`df = df.reindex(columns=expected_cols)`). - For truly large-scale merges (hundreds of files, GBs each), skip Python-level looping entirely and use `cat file1.csv file2.csv ... > merged.csv` at the OS level (when schemas are guaranteed identical) — it's dramatically faster than any Python approach.

---

7. Find the top 10 most frequent values from a huge dataset

```python
from collections import Counter

def top_n(filepath, n=10):
    counter = Counter()
    with open(filepath) as f:
```

```
        for line in f:
            counter[line.strip()] += 1
    return counter.most_common(n)
```

Architect notes - `Counter.most_common(n)` internally uses a **heap of size n**, not a full sort of all unique values — O(k log n) instead of O(k log k) for the full-sort alternative. This distinction matters when there are millions of unique values. - If the *entire dataset* (not just unique-value counts) doesn't fit in memory, this becomes a **heavy-hitters / streaming top-K problem** — the exact-count approach above still works as long as the *number of distinct values* fits in memory, even if the raw stream is unbounded. If even distinct-value cardinality is too large, approximate structures (Count-Min Sketch, Space-Saving algorithm) trade exactness for bounded memory. - For distributed-scale data, this is a one-line operation in Spark (`df.groupBy(col).count().orderBy(desc("count")).limit(10)`) — know when to stop writing custom Python and reach for the right engine.

---

8. Write a retry mechanism for an API call with exponential backoff

```
import time
import random
import requests

def call_with_retry(url, max_retries=5, base_delay=1, max_delay=30):
    for attempt in range(max_retries):
        try:
            response = requests.get(url, timeout=5)
            response.raise_for_status()
            return response.json()
        except requests.exceptions.RequestException as e:
            if attempt == max_retries - 1:
                raise
            delay = min(base_delay * (2 ** attempt), max_delay)
            jitter = random.uniform(0, delay * 0.1)
            time.sleep(delay + jitter)
    raise RuntimeError("Unreachable")
```

Architect notes - **Jitter is not optional** in production — without it, many clients retrying in lockstep after the same failure re-hammer the server at the same moments (the "thundering herd" problem), which is often what caused the original failure. - Retries should be reserved for **transient, idempotent** failures (timeouts, 429, 503). Retrying a non-idempotent POST that partially succeeded can duplicate side effects — check idempotency before blindly wrapping retry logic around any call. - In production code, don't hand-roll this — use a battle-tested library (`tenacity` in Python, or a service mesh's built-in retry policy) that also handles circuit-breaking (stop retrying entirely once a downstream is clearly down, rather than retrying every single call and making the outage worse). - Distinguish `max_retries` (attempt cap) from `max_delay` (backoff cap) — uncapped exponential backoff on attempt 20 will happily try to sleep for hours.

---

9. Validate incoming data (NULLs, duplicates, data types) before loading

```python
from pydantic import BaseModel, ValidationError, field_validator
from typing import Optional
import pandas as pd

class Record(BaseModel):
    id: int
    email: str
    amount: float

    @field_validator("email")
    @classmethod
    def email_not_empty(cls, v):
        if not v or "@" not in v:
            raise ValueError("invalid email")
        return v

def validate_batch(df: pd.DataFrame):
    valid, errors = [], []
    seen_ids = set()

    for i, row in df.iterrows():
        if row["id"] in seen_ids:
            errors.append((i, "duplicate id"))
            continue
        try:
            record = Record(**row.to_dict())
            valid.append(record)
            seen_ids.add(row["id"])
        except ValidationError as e:
            errors.append((i, str(e)))

    return valid, errors
```

Architect notes - Prefer a **schema-driven validator** (Pydantic, `jsonschema`, Great Expectations, or `pandera` for DataFrame-native validation) over ad hoc `if/else` checks — it's declarative, self-documenting, and reusable across ingestion points. - Separate **structural validation** (types, nullability, format) from **business validation** (email domain allow-lists, amount within expected range) — they evolve at different rates and often live in different layers of the pipeline. - Decide the failure policy explicitly: **reject the whole batch** on any bad row (strict, safer for financial data) vs. **quarantine bad rows and load the rest** (lenient, better for high-volume low-criticality data). Log rejected rows somewhere queryable (a dead-letter table), don't just drop them silently. - At scale, row-by-row Python validation (`iterrows()`) is slow — vectorized validation (`pandera`, or vectorized pandas boolean masks for null/dup checks) will outperform this by orders of magnitude on large DataFrames.

---

10. Process thousands of files in parallel and generate a processing log

```
import concurrent.futures
import logging
import glob

logging.basicConfig(
    filename="processing.log",
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s"
)

def process_file(filepath):
    try:
        # ... actual processing logic ...
        logging.info(f"SUCCESS: {filepath}")
        return (filepath, "success", None)
    except Exception as e:
        logging.error(f"FAILED: {filepath} | {e}")
        return (filepath, "failed", str(e))

def process_all(folder, max_workers=8):
    files = glob.glob(f"{folder}/*")
    results = []
    with concurrent.futures.ProcessPoolExecutor(max_workers=max_workers) as
executor:
        futures = {executor.submit(process_file, f): f for f in files}
        for future in concurrent.futures.as_completed(futures):
            results.append(future.result())
    return results
```

Architect notes - `ProcessPoolExecutor` vs `ThreadPoolExecutor`: use **processes** for CPU-bound file processing (parsing, transforms — bypasses the GIL); use **threads** if the work is I/O-bound (network calls, disk waits dominate, GIL isn't the bottleneck). - Python's built-in `logging` module is thread-/process-safe for writing to a single file **from threads**, but writing to the same file from multiple **processes** can interleave/corrupt lines — use a `QueueHandler` + a single logging listener process, or have each worker log to its own file and merge afterward. - `max_workers` should reflect actual bottleneck: CPU cores for CPU-bound work, or a tuned concurrency limit for I/O-bound work against a rate-limited downstream (API, database) — unbounded parallelism against a fixed-capacity dependency just shifts the bottleneck into timeouts and retries. - For "thousands of files" at real scale, consider whether this belongs in a proper **workflow orchestrator** (Airflow, Prefect, Dagster) instead of a single script — you get retry-per-task, dead-letter handling, and visibility into partial failures for free, rather than re-implementing it in `concurrent.futures`.

Cross-Cutting Architect Takeaways

- **Window functions over subqueries**: almost every "Nth", "top-N per group", and "running total" SQL question collapses to the same window-function idiom — `PARTITION BY` + `ORDER BY` + a ranking or aggregate function. Recognize the shape once, apply everywhere.
- **Streaming over loading everything into memory**: the Python answers that scale (word count, dedup, retries, parallel processing) all avoid materializing the entire dataset at once. The naive one-liner and the production-grade version usually differ exactly on this axis.
- **Know when to stop hand-rolling**: retries → use `tenacity`; validation → use `pydantic`/`pandera`; huge-scale joins/aggregations → use Spark/warehouse SQL, not pandas; orchestrating thousands of parallel jobs → use a workflow engine, not a `for` loop. A senior engineer's edge is often knowing which of these to *not* write from scratch.